

ESE 650, SPRING 2025

HOMEWORK 3

JINHAO YOU [18330723]

(a) Environment Setup. We model a 10×10 grid-world MDP with coordinates (x, y) , where $(0, 0)$ is at the bottom-left. All border cells and cell $(3, 2)$ are *sticky obstacles*; the robot at $(1, 1)$ (start, blue) must reach $(8, 1)$ (goal, green). **Transition model.** At each non-obstacle, non-goal cell the

robot chooses one of four controls $u \in \{N, S, E, W\}$. The stochastic outcome is

$$P(\text{intended} \mid u) = 0.7, \quad P(\text{left of } u) = 0.1, \quad P(\text{right of } u) = 0.1, \quad P(\text{stay}) = 0.1.$$

If the resulting cell is out-of-bounds or an obstacle, the robot stays in place. Obstacles are sticky: once entered, the robot remains there permanently. **Reward structure.**

- Solution 1** (Policy Iteration).
- Goal state $(8, 1)$: absorbing, reward = 0 (“not move” action).
 - Obstacle cell: reward = -10 per time step.
 - All other cells: reward = -1 per action taken.
 - Reaching the goal: the terminal reward of $+10$ is implicitly encoded as the 0-cost absorbing state whose discounted value far exceeds any other cell.

Discount factor $\gamma = 0.9$.

Implementation. The transition matrix $T_{x,x'}(u) = P(x' \mid x, u)$ is a 100×100 matrix for each action. The run-time cost $q(x, u, x')$ depends on the current state x as described above. Policy evaluation is performed via exact linear solve $(I - \gamma T_\pi)^{-1} R_\pi$.

(b) Policy Evaluation: $\pi^{(0)} = \text{always East}$. Figure 1 shows the value function $J^{\pi^{(0)}}(x)$ under the initial policy “always go East.”

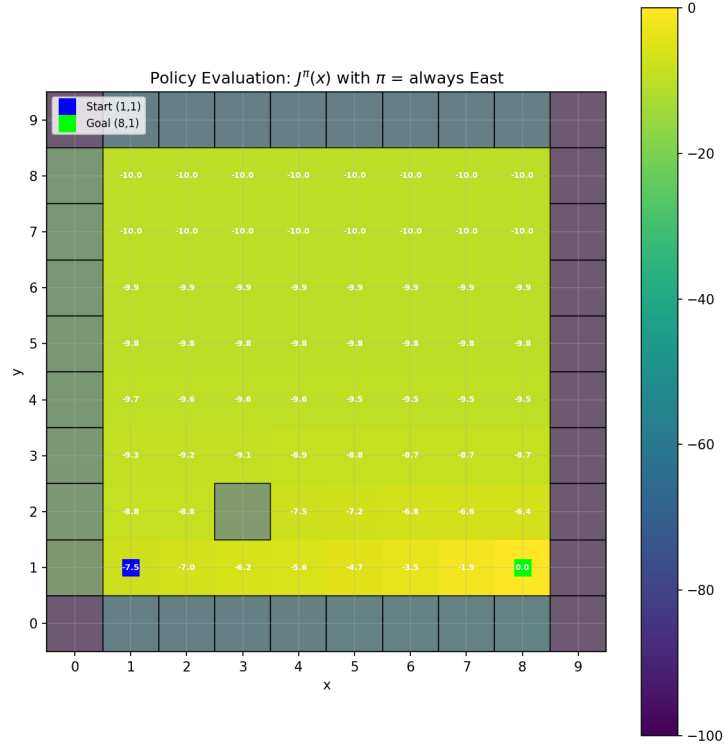
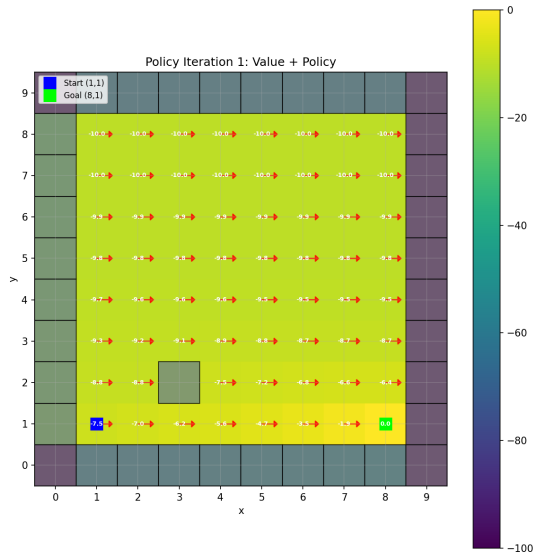


FIGURE 1. Value function heatmap for $\pi^{(0)}$ (always East). The start cell $(1, 1)$ has $J = -7.54$; the goal $(8, 1)$ has $J = 0$. Row 1 shows a clear gradient toward the goal, while upper rows have uniformly poor values (≈ -10) because the East-only policy cannot navigate downward.

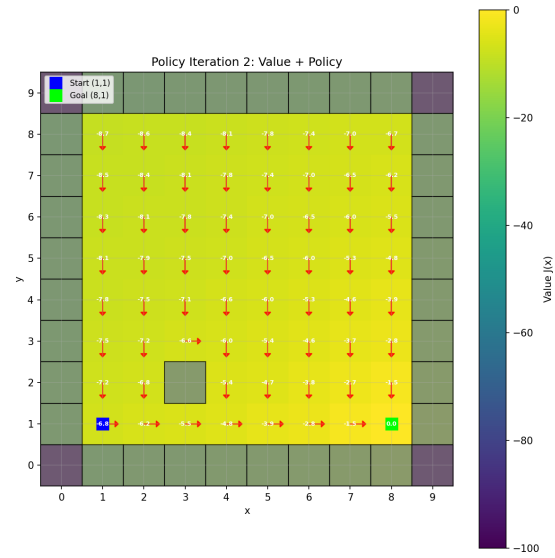
Key observations:

- Along row $y = 1$, the value increases monotonically from -7.54 at $(1, 1)$ to 0 at the goal $(8, 1)$, confirming that the “always East” policy is effective along this corridor.
- For cells with $y \geq 2$, values are nearly -10 because going East cannot decrease y toward the goal row; the robot is essentially trapped.
- Obstacle cells (gray) have deeply negative values (~ -100) due to the $-10/\text{step}$ penalty accumulated over the infinite horizon.

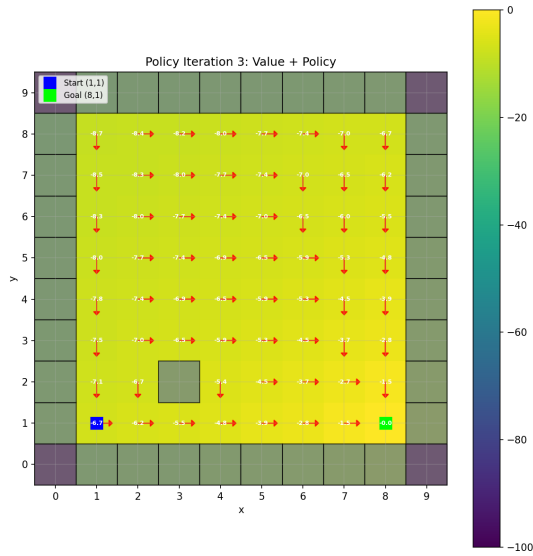
(c) Policy Iteration (4 iterations). Starting from $\pi^{(0)}$ (always East), we iteratively perform policy evaluation and policy improvement. Figures 2a–2d show the first four iterations, and Figure 3 shows the final converged policy.



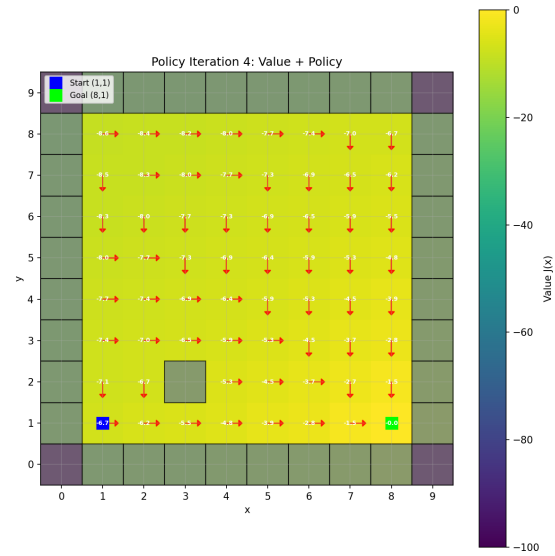
(A) Iteration 1 (before first improvement): the “always East” policy. 84 cells changed after improvement.



(B) Iteration 2: after first improvement. Arrows now point toward the goal. 31 cells changed.



(C) Iteration 3: policies largely converged. 18 cells changed.



(D) Iteration 4: near convergence. Only 6 cells changed.

FIGURE 2. Policy iteration: value heatmap + policy arrows (red) for 4 iterations.

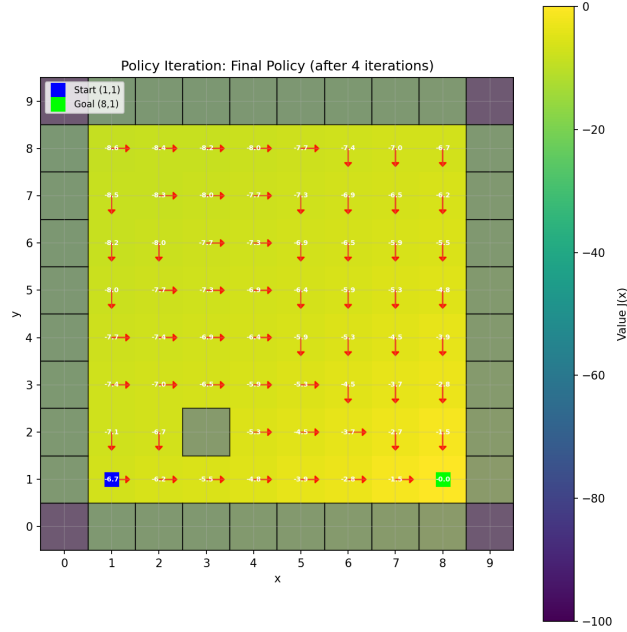


FIGURE 3. Final policy after 4 improvement steps. $J^*(1, 1) = -6.74$, improved from -7.54 under the initial East-only policy. The arrows show that the optimal policy directs the robot south-east toward the goal, routing around the obstacle at $(3, 2)$.

Convergence. The number of policy changes per iteration is $84 \rightarrow 31 \rightarrow 18 \rightarrow 6$, indicating rapid convergence. The value at the start improves from -7.54 (East-only) to -6.74 (optimized).

Bellman equation. The optimal value satisfies

$$J^*(x) = \min_{u \in U} \mathbb{E}_{x'} [q(x, u, x') + \gamma J^*(x')].$$

Policy iteration alternates between evaluating $J^{\pi^{(k)}}$ (linear solve) and improving $\pi^{(k+1)}(x) = \arg \min_u Q^{\pi^{(k)}}(x, u)$.

(a) Data Loading and COLMAP. We use the provided `transforms_colmap.json` file which contains camera intrinsics and extrinsics for 100 LEGO images. **Camera parameters.** The original images are 800×800 . We resize to 100×100 for efficient CPU training. The focal length is computed from the horizontal FOV:

$$f_{\text{orig}} = \frac{W_{\text{orig}}}{2 \tan(\theta_x/2)}, \quad f = f_{\text{orig}} \times \frac{100}{800} = 138.89 \text{ px.}$$

where $\theta_x = \text{camera_angle_x}$ from the JSON file. After resizing: $H = 100$, $W = 100$, $f = 138.89$.

(b) Implementation of the NeRF. We implement four key functions: **1. get_rays** (H, W, f, T_w^c) .

For each pixel (i, j) , we compute the ray direction in camera coordinates using the pinhole model:

$$\mathbf{d}_{\text{cam}} = \left(\frac{j - W/2}{f}, -\frac{i - H/2}{f}, -1 \right).$$

The NeRF coordinate convention is $+X$ right, $+Y$ up, $+Z$ back. The ray direction and origin are transformed to world coordinates via the 4×4 camera-to-world matrix T_w^c :

$$\mathbf{o} = T_w^c[:3, 3], \quad \mathbf{d} = \frac{R \cdot \mathbf{d}_{\text{cam}}}{\|R \cdot \mathbf{d}_{\text{cam}}\|}.$$

2. sample_points_from_rays. Given near bound s_{near} and far bound s_{far} , we perform stratified sampling along each ray:

$$t_k = s_{\text{near}} + \frac{k}{N_{\text{sample}}} (s_{\text{far}} - s_{\text{near}}) + \xi_k, \quad \xi_k \sim \mathcal{U}\left(0, \frac{s_{\text{far}} - s_{\text{near}}}{N_{\text{sample}}}\right),$$

yielding 3D sample points $\mathbf{x}_k = \mathbf{o} + t_k \mathbf{d}$. **3. positional_encoding.** We map 3D coordinates to a higher-dimensional space using sinusoidal functions:

$$\gamma(\mathbf{x}) = (\mathbf{x}, \sin(2^0 \mathbf{x}), \cos(2^0 \mathbf{x}), \dots, \sin(2^{L-1} \mathbf{x}), \cos(2^{L-1} \mathbf{x}))$$

with $L = 10$ frequencies. The output dimension is $(1 + 2 \times 10) \times 3 = 63$. **4. volume_rendering.**

Given the predicted radiance field $(\sigma_k, \mathbf{c}_k)$ at each sample point (with $\sigma = \text{ReLU}(\cdot)$ and $\mathbf{c} = \text{sigmoid}(\cdot)$), we compute the rendered color via alpha compositing:

$$\delta_k = t_{k+1} - t_k, \tag{1}$$

$$\alpha_k = 1 - e^{-\sigma_k \delta_k}, \tag{2}$$

$$T_k = \prod_{j=1}^{k-1} (1 - \alpha_j), \tag{3}$$

$$\hat{C} = \sum_{k=1}^{N_{\text{sample}}} T_k \alpha_k \mathbf{c}_k. \tag{4}$$

(d) Network Training. Architecture. TinyNeRF: 4-layer MLP (Linear–ReLU–Linear–ReLU–Linear–ReLU–Linear), input dimension 63 (positional encoding), hidden dimension $h_{\text{dim}} = 32$, output dimension 4 (RGB + σ). Total parameters: 4,292.

Hyperparameters.

Solution 2 (Building a NeRF).

- $N_{\text{sample}} = 32, h_{\text{dim}} = 32$
- $s_{\text{near}} = 2.0, s_{\text{far}} = 6.0$
- Learning rate: 5×10^{-3} (Adam)
- Training iterations: 1000
- Image resolution: 100×100

../p3/training_loss.png

FIGURE 4. Training loss (MSE) as a function of iteration.

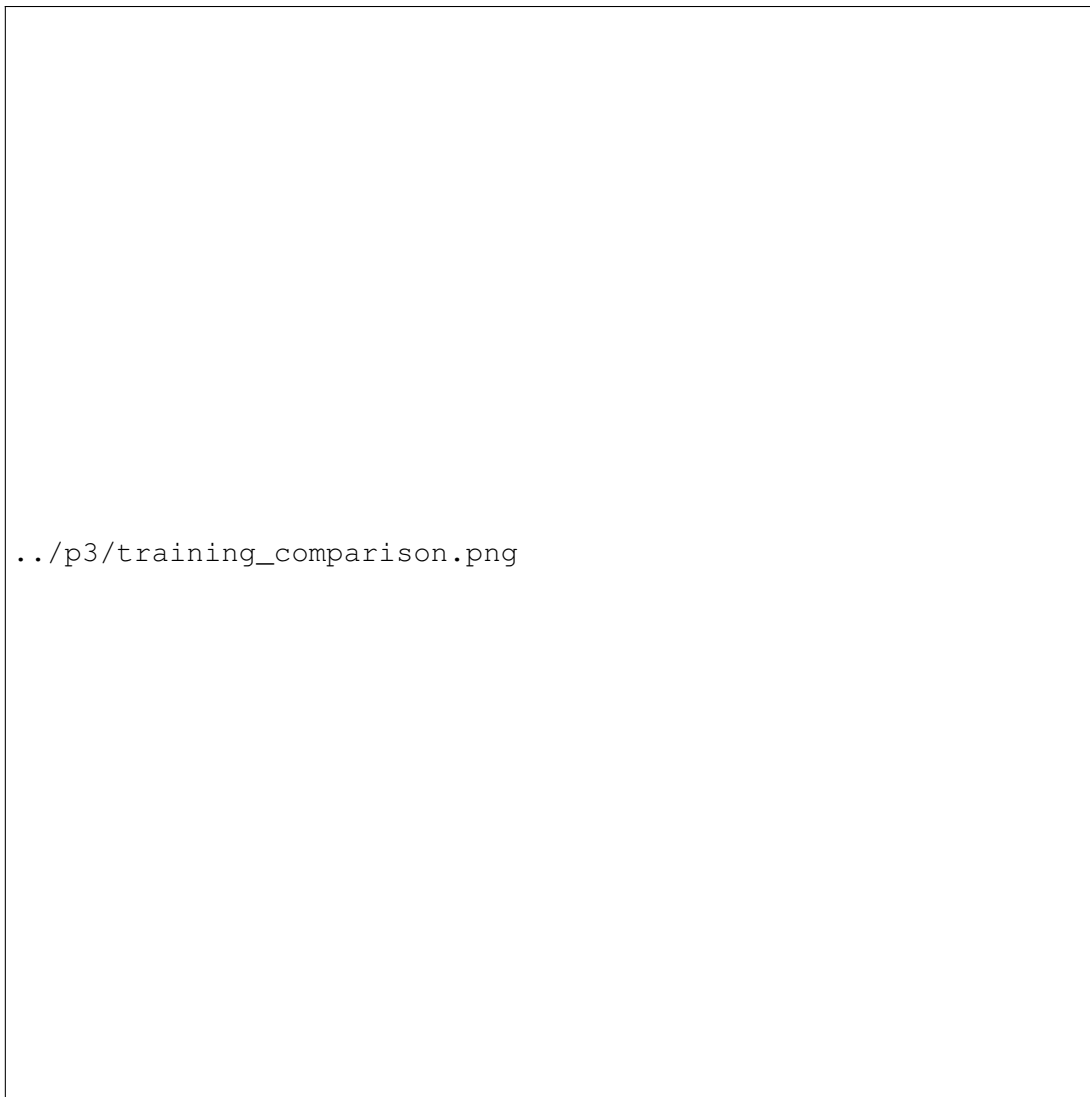


FIGURE 5. Training set comparison: ground truth (top) vs. NeRF prediction (bottom) for 6 sampled viewpoints.

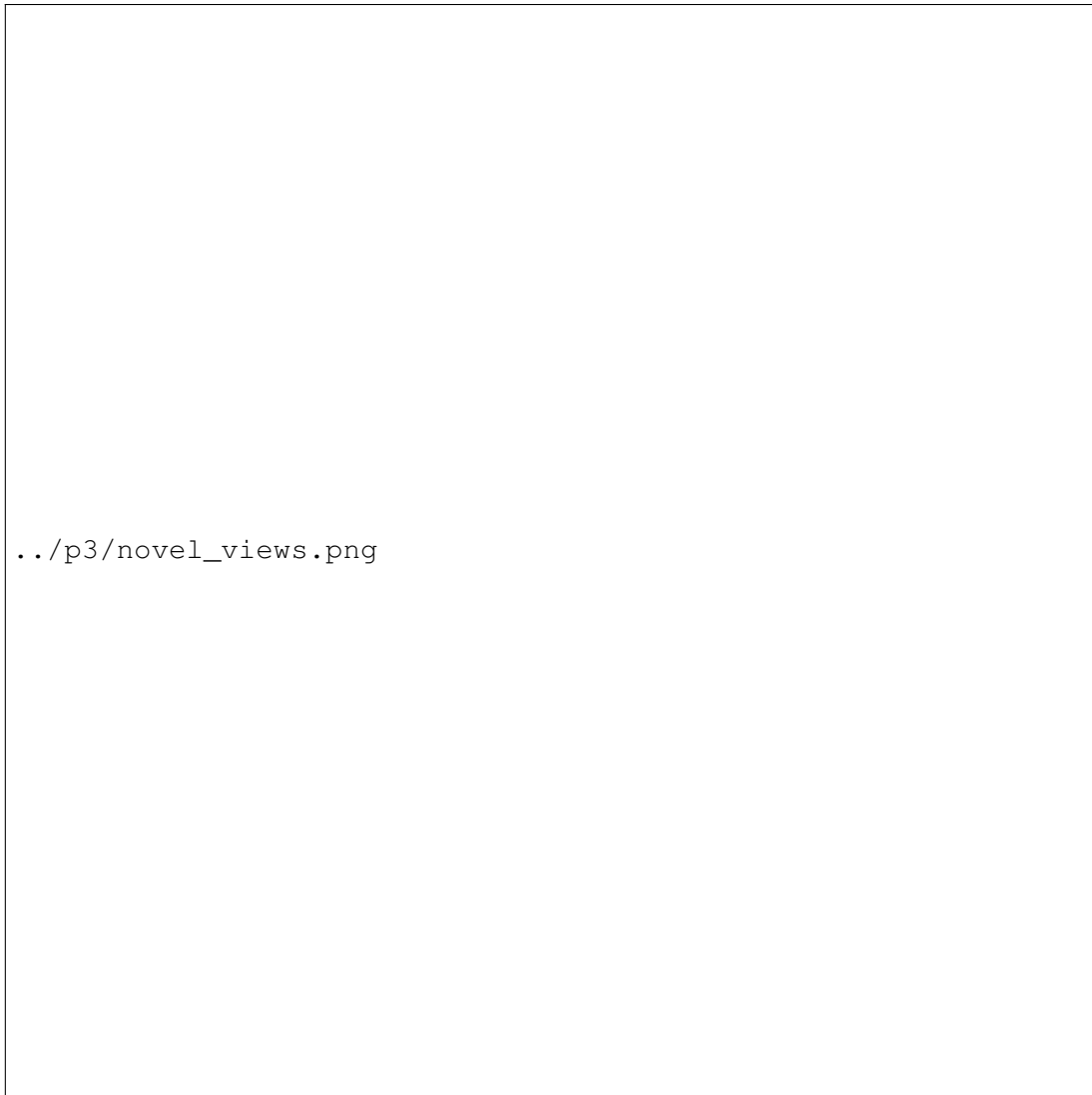


FIGURE 6. Novel viewpoint rendering: 5 interpolated poses between training views.

(e) Inference. Novel views are generated by linearly interpolating between two training poses. The NeRF produces coherent geometry from nearby viewpoints, but may show artifacts (blurriness, color drift) for poses far from the training distribution, which is expected given the simplified architecture and limited training.